CI/CD Pipeline

What is CI/CD?

CI/CD stands for **Continuous Integration** and **Continuous Deployment/Delivery**. It is a software development practice that enables teams to deliver code changes frequently and reliably through automation.

- **Continuous Integration (CI):** Developers regularly merge their code changes into a shared repository, triggering automated builds and tests.
- **Continuous Delivery (CD):** The validated code is automatically pushed to a staging environment.
- **Continuous Deployment (CD):** The code is automatically deployed to production after passing all tests.

Benefits of CI/CD

- Faster Delivery
- Improved Code Quality
- Early Detection of Bugs
- Reduced Manual Effort
- Better Collaboration

CI/CD Pipeline Stages

1. Source Stage

- Developers commit code to version control (e.g., GitHub, GitLab).

2. Build Stage

- Code is compiled and dependencies are installed.

CI/CD Pipeline

- Example tools: Maven, Gradle, npm

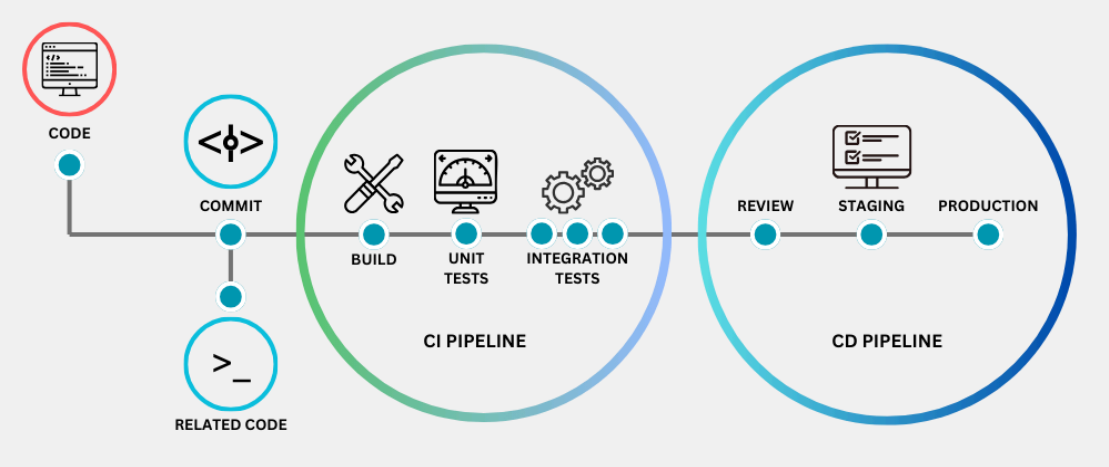
3. Test Stage

- Automated unit and integration tests are run.
- Tools: JUnit, Selenium, PyTest

4. Deploy Stage

- Code is deployed to staging or production environments.
- Tools: Docker, Kubernetes, Ansible

Sample CI/CD Pipeline Diagram



Tools Used in CI/CD

Purpose	Tools
n Control	Git, GitHub, GitLab
Server	s, GitHub Actions, CircleCI
nerization	Docker

CI/CD Pipeline

stration	netes
uration	e, Terraform
oring	theus, Grafana

Best Practices

- Commit small and frequent changes
- Automate as much as possible
- Maintain test coverage
- Use staging environments
- Monitor deployments

Conclusion

CI/CD pipelines streamline development, testing, and deployment. Adopting CI/CD can lead to faster releases, fewer bugs, and higher productivity for development teams.